

American Sign Language Recognition

Group Assignment

A paper submitted in partial fulfillment of the requirements for the course Machine Learning and Deep Learning

Kühne Logistics University

Students:

Daria Fedorova (990215083)

Marcela Santos (990215807)

William Merino-Galindo (990215797)

Supervisor: Prof. Dr. Asvin Goel

Date of Submission: April 29, 2026

GitHub Link: [American Sign Language Recognition](#)

Table of Contents

AI USE STATEMENT	2
I. INTRODUCTION	4
II. DATASET OVERVIEW	5
III. MODELS IMPLEMENTED	7
IV. MODELS COMPARISON	11
V. CONVOLUTIONAL AUTOENCODER AND ANOMALY DETECTION	13
VI. REFLECTION AND RETROSPECTIVE	14
VII. CONCLUSION	15

Figures

Figure 1 – ASL dataset image	4
Figure 2 – EDA chart	5
Figure 3 – Best vs Worst Image Reconstructions	6
Figure 4 – Best vs Worst Image Reconstructions	7
Figure 5 - Confusion Matrix LeNet-5	8
Figure 6 – Training and Validation Loss and Accuracy Curves for the Custom CNN	10
Figure 7 – Normalized Confusion Matrix for Custom CNN	11
Figure 8 – Test Accuracy Comparison	12
Figure 9 – Training History Comparison	13
Figure 10 – Model Comparison Table	13

AI Use Statement

Artificial intelligence (AI) tools were used in the preparation of this project for the following purposes: (1) code suggestions and debugging assistance during implementation of both CNN models; (2) grammar and clarity review of written sections; and (3) generation of comparative performance commentary. All algorithmic design decisions, experimental configurations (parameters), and interpretations presented in this report are the original work of the author. The use of AI tools is acknowledged in accordance with academic integrity guidelines.

I. Introduction

Sign language is the primary means of communication for millions of deaf and hard-of-hearing individuals worldwide. Despite its importance, there remains a significant communication barrier between sign language users and those who are unfamiliar with it. Automated sign language recognition systems have the potential to bridge this gap by enabling real-time translation of hand gestures into text or speech, making communication more accessible and inclusive. Recent advances in deep learning, particularly in convolutional neural networks, have made such systems increasingly practical and accurate.

This project addresses the task of static American Sign Language (ASL) letter recognition using machine learning and deep learning techniques. The goal is to develop and evaluate models capable of correctly classifying images of hand gestures corresponding to the 24 static letters of the ASL alphabet. The letters J and Z are excluded from this task, as they require motion to be performed and therefore cannot be represented as static images. The project explores multiple approaches, ranging from a classical CNN baseline to a deeper custom architecture and an unsupervised autoencoder, to assess how different methods handle the complexity of the classification task.

The dataset used throughout this project is the Sign Language MNIST, a well-established benchmark dataset modeled after the original MNIST dataset. It consists of 28×28 grayscale images of hand gestures, with 27,455 training samples and 7,172 test samples spread across 24 classes. The structured and standardized nature of this dataset makes it well-suited for evaluating and comparing different model architectures, while still presenting meaningful classification challenges due to the visual similarity between certain ASL letters.



Figure 1 – ASL dataset image

II. Dataset Overview

Before training the models, an exploratory data analysis (EDA) was conducted to better understand the structure and characteristics of the dataset. As shown in Figure 1, the class distribution is relatively balanced across all ASL letters, which helps prevent bias toward specific classes during training. The train, validation, and the test split ensure that the model is evaluated on unseen data while maintaining sufficient data for learning.

Additionally, the mean image per class highlights that each letter has a distinct visual pattern, supporting the suitability of convolutional neural networks for this task. The pixel intensity distribution further confirms that the images contain sufficient variation for effective feature extraction. Finally, the random sample images illustrate the consistency of hand positioning across samples, while also showing slight variations that justify the use of data augmentation.



Figure 2 – EDA chart

To further explore the characteristics of the dataset, the convolutional autoencoder was used to analyze how well individual images can be reconstructed, providing an additional perspective on data quality and consistency. As shown in Figure 3, the best and worst reconstructed images reveal that samples with clear, well-centered hand positioning and high contrast are reconstructed with very low error, confirming that these images contain clean and consistent visual information. In contrast, the images with the highest reconstruction error tend to correspond to letters with more complex or ambiguous hand configurations, suggesting that these samples are inherently more difficult for the model to represent compactly. This aligns with the classification challenges observed in the supervised models, where the same letters tended to produce more errors.

Figure 4 further illustrates the dataset's sensitivity to image corruption through the corrupted examples of analysis. When images were subjected to heavy noise and random masking, the autoencoder produced significantly higher reconstruction errors, demonstrating that the dataset's visual structure is both learnable and distinct enough to be disrupted by artificial degradation. This reinforces the importance of the data augmentation strategy applied during supervised training, as even minor variations in brightness and noise can affect the model's ability to generalize. Together, these figures confirm that the Sign Language MNIST dataset is well-structured and consistent, while also highlighting the areas where image quality and visual ambiguity present the greatest challenge.

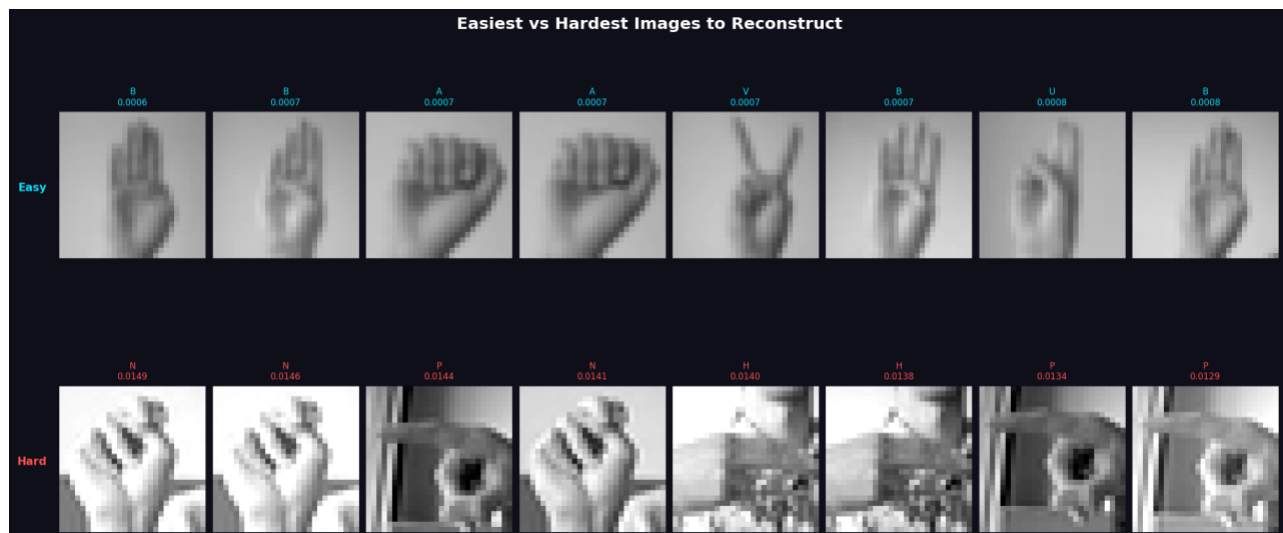


Figure 3 – Best vs Worst Image Reconstructions

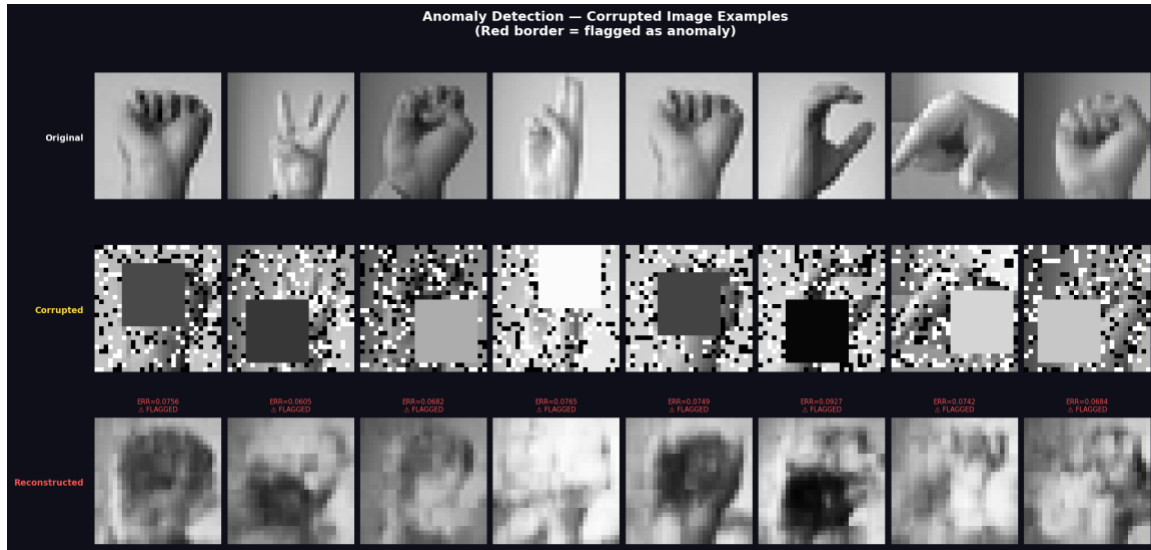


Figure 4 – Best vs Worst Image Reconstructions

III. Models implemented

Several convolutional neural network (CNN) architectures were implemented and evaluated to address the sign language recognition classification task. The selection of models was designed to provide a meaningful comparison between a well-established baseline and more advanced approaches, allowing for an assessment of how architectural complexity influences performance. In particular, the study includes a classic LeNet-5 model, representing a standard and lightweight CNN, alongside more complex custom-designed networks.

LeNet- 5 Model

In this project, a CNN based on the LeNet-5 architecture was implemented as one of the baseline models for classifying American Sign Language (ASL) hand gestures. The model was adapted to process 28×28 grayscale images, which corresponds directly to the format of the Sign Language MNIST dataset used in this study.

The architecture consists of a feature extraction stage followed by a classification stage. The feature extractor includes two convolutional layers with 5×5 filters, each followed by a ReLU (Rectified Linear Unit) activation function and average pooling. The ReLU activation introduces non-linearity by setting all negative values to zero while keeping positive values unchanged, allowing the network to learn more complex patterns beyond simple linear relationships. These layers progressively learn visual features, starting from simple edges and contours to more complex hand shapes. The resulting feature maps are then flattened and passed through three fully connected layers, which reduce the feature representation and produce the final class scores (logits).

LeNet-5 is well suited for this task due to the standardized nature of the dataset, where images are grayscale, centered, and of fixed size. These characteristics are similar to those of the original MNIST dataset, making the architecture effective for capturing the spatial patterns of hand gestures.

The model was chosen primarily for its simplicity, efficiency, and its role as a standard architecture in image recognition tasks. LeNet-5 is one of the earliest successful CNN and is widely used as a reference model in image classification problems, particularly for small grayscale images. Its relatively shallow structure makes it easy to interpret and fast to train, while still providing reliable performance. As a result, LeNet-5 serves as a strong baseline for evaluating more complex models in this project.

The performance of the LeNet-5 model, as shown in the confusion matrix (Figure 5), reflects these architectural characteristics.

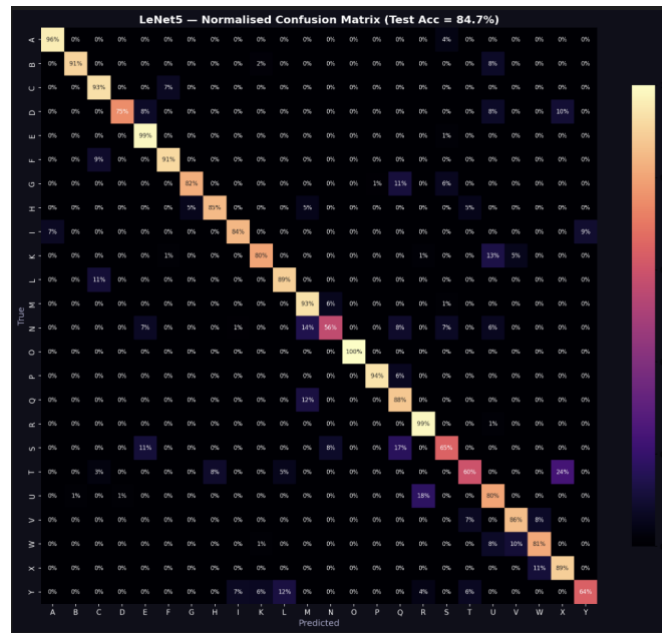


Figure 5 - Confusion Matrix LeNet-5

The model achieves an overall test accuracy of approximately 84.7%, with strong diagonal dominance indicating that many classes are correctly classified with high confidence. Letters such as E, O, R, and A are recognized particularly well, demonstrating that the model is effective at capturing clear and distinct visual patterns. However, the confusion matrix also reveals consistent misclassifications for certain letters, including N, S, T, and Y, where accuracy is noticeably lower. These errors primarily occur between visually similar hand gestures, suggesting that the model struggles to distinguish subtle differences in finger positioning and hand configuration. This limitation can be directly linked to the relatively shallow structure of LeNet-5, which, while efficient, lacks the depth required to learn more fine-grained and complex feature representations.

Therefore, although LeNet-5 provides solid and reliable baseline performance, the confusion matrix highlights the need for more advanced architectures to improve classification of visually similar signs.

Custom CNN Model

In addition to the baseline LeNet-5 architecture, a deeper custom Convolutional Neural Network (CNN) was developed to improve classification performance for American Sign Language (ASL) hand gestures. The design of this model aimed to address the limitations of simpler architectures by increasing depth and incorporating modern techniques such as batch normalization and dropout, allowing the network to learn more complex and fine-grained visual features.

The Custom CNN architecture follows a standard CNN structure, consisting of a feature extraction stage followed by a classification stage. The model includes three convolutional blocks; each composed of two convolutional layers with 3×3 filters. Each convolutional layer is followed by batch normalization and a ReLU activation function. As previously mentioned, in the LeNet-5 model, the batch normalization helps stabilize and accelerate training by normalizing intermediate feature distributions, while ReLU introduces non-linearity, enabling the model to capture more complex patterns in the data.

After each convolutional block, max pooling is applied to reduce the spatial dimensions of the feature maps, allowing the model to focus on the most important features while reducing computational complexity. In addition, dropouts are applied after each block with increasing rates, which helps prevent overfitting by randomly deactivating a proportion of neurons during training.

As the network progresses deeper, the number of feature maps increases from 32 to 64 and finally to 128. This allows the model to move from detecting simple features such as edges and contours to more complex representations of hand shapes and finger configurations. The extracted features are then flattened and passed through a fully connected classification stage, consisting of one hidden layer with 256 neurons and a final output layer corresponding to the number of ASL classes.

The model was trained using the Adam optimizer with a learning rate of 0.001 and cross-entropy loss. To further improve training performance, a learning rate scheduler was implemented, which reduces the learning rate when the validation loss stops improving. Additionally, early stopping was applied to prevent overfitting by terminating training when validation performance no longer improves over a set number of epochs. Light data augmentation was also applied during training, including small brightness variations and Gaussian noise, in order to improve generalization while preserving the semantic meaning of the hand signs.

In addition to the final model configuration, different learning rates were tested to evaluate their impact on model performance. In particular, an initial learning rate of 0.005 was experimented with, but it resulted in slightly lower performance compared to the final chosen value of 0.001. The model trained with a learning rate of 0.005 achieved an overall test accuracy of approximately 98.12%.

The classification results with the higher learning rate also showed slightly more variability across certain classes. For example, lower recall values were observed for letters such as N and T, and reduced precision for classes like Q and X, indicating that the model struggled more with correctly distinguishing some visually similar hand gestures. This suggests that the higher learning rate may have caused the model to converge less smoothly, potentially overshooting optimal solutions during training.

As a result, a learning rate of 0.001 was selected for the final model, as it provided more stable training behavior and consistently higher overall performance. This highlights the importance of hyperparameter tuning, particularly the learning rate, in achieving optimal model performance.

The training process of the Custom CNN model is illustrated in Figure 6, which shows both the loss and accuracy curves over the training epochs. The loss curves demonstrate a rapid decrease in training loss during the initial epochs, followed by a gradual stabilization, indicating that the model converges efficiently. Similarly, the accuracy curves show a sharp increase in performance, with both training and validation accuracy reaching very high levels early in the training process.

The close alignment between training and validation curves suggests that the model generalizes well and does not suffer from significant overfitting. Furthermore, the early stabilization of validation accuracy supports the effectiveness of the learning rate scheduler and early stopping mechanism in preventing unnecessary training beyond convergence. The validation accuracy reaching near 100% at an early stage may also indicate that the dataset is relatively structured and less noisy compared to real-world image datasets.

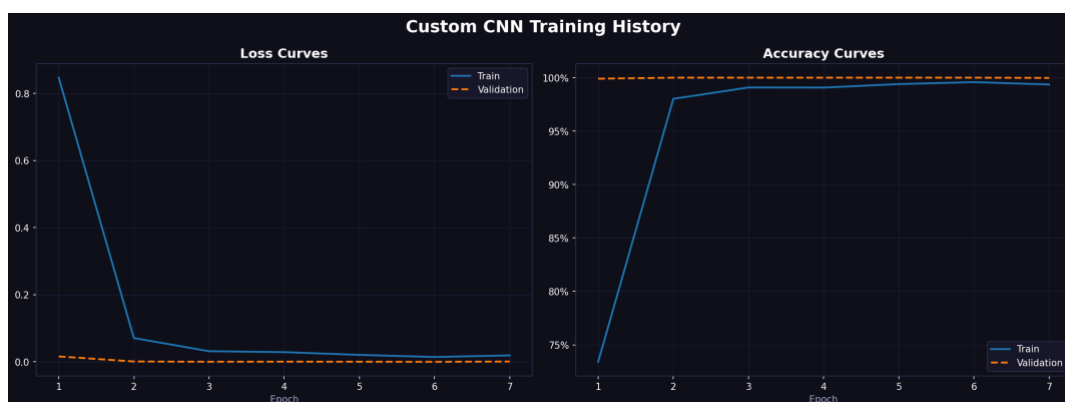


Figure 6 – Training and Validation Loss and Accuracy Curves for the Custom CNN

The performance of the Custom CNN model, as shown in the confusion matrix (Figure 7), demonstrates a high level of accuracy in classifying ASL hand gestures, achieving a test accuracy of 98.65%. The confusion matrix exhibits strong diagonal dominance, indicating that most classes are classified correctly with very high accuracy. Many letters, such as D, E, H, and I, are recognized almost perfectly, reflecting the model's ability to learn highly discriminative features. Some minor confusion remains for certain letters, including G, N, and K. These errors are primarily associated with visually similar hand gestures, where small differences in finger positioning are difficult to capture. However, these misclassifications are limited and concentrated, indicating that the model performs consistently well across all classes.

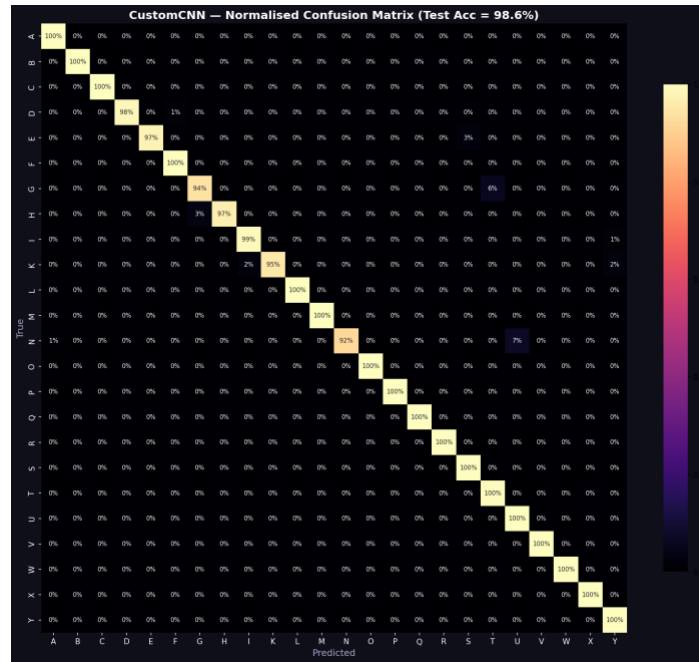


Figure 7 – Normalized Confusion Matrix for Custom CNN

Overall, the Custom CNN model demonstrates very strong performance for the ASL classification task, effectively learning detailed feature representations, and achieving high accuracy across nearly all classes.

IV. Models Comparison

The performance of the implemented models: LeNet-5, CustomCNN_f0_5, and CustomCNN_f0_1, was evaluated using test accuracy, training history, and model complexity. The results demonstrate a clear performance gap between the baseline LeNet-5 model and the more advanced custom CNN architectures.

As shown in Figure 5, LeNet-5 achieves a test accuracy of 84.68%, whereas both custom models significantly outperform it, reaching 98.12% for CustomCNN_f0_5 and 98.02% for CustomCNN_f0_1. We must note that all these were tested several times, therefore each run gave a different result. In the end, as previously mentioned above in Custom CNN model explanation, CustomCNN_f0_1 behaved best with 96.65 accuracy. Once we re-ran everything, the results changed, but overall, in each run CustomCNN_f0_1 behaved better than CustomCNN_f0_5.

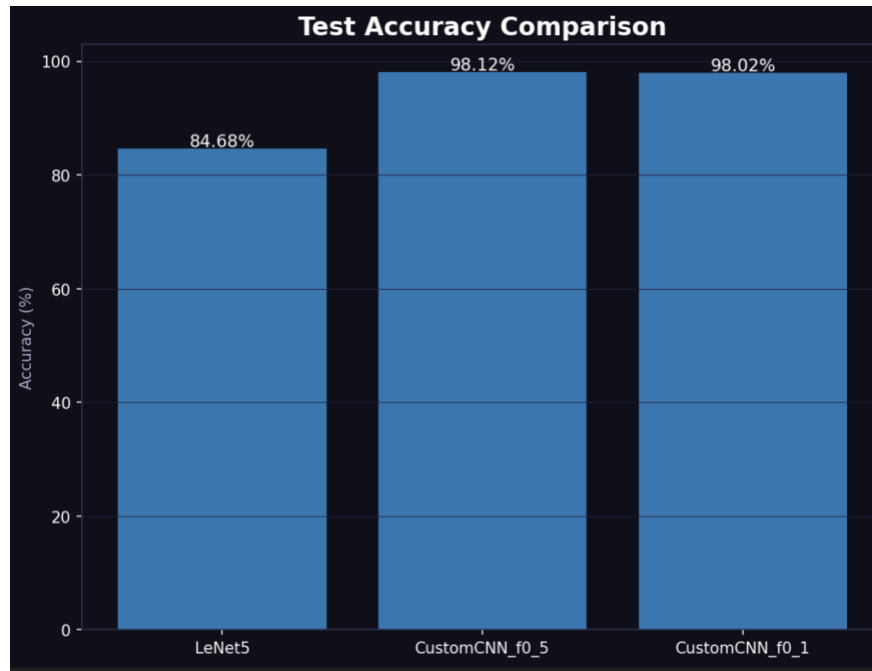


Figure 8 – Test Accuracy Comparison

This improvement of over 13% highlights the advantage of using deeper architectures for this task. While all models achieved near-perfect validation accuracy during training, the lower test accuracy of LeNet-5 indicates weaker generalization compared to the custom models.

The training history further supports this observation, as shown in Figure 9. The custom CNN models exhibit a much faster convergence, with both training and validation accuracy reaching close to 100% within the first few epochs. In contrast, LeNet-5 shows a slower learning curve, requiring more epochs to approach similar training accuracy levels. Additionally, the loss curves indicate that the custom models achieve lower and more stable loss values earlier in training, reflecting more efficient optimization and better feature learning.



Figure 9 – Training History Comparison

Model complexity also plays a key role in explaining these differences. LeNet-5 contains approximately 45,616 parameters, while each custom CNN model has 588,664 parameters, making them significantly more expressive.

Model comparison:				
Model	Parameters	Best Val Accuracy	Test Accuracy	
LeNet5	45616	100.0	84.68	
CustomCNN_f0_5	588664	100.0	98.12	
CustomCNN_f0_1	588664	100.0	98.02	

Figure 10 – Model Comparison Table

This increased capacity allows the custom models to learn more detailed and discriminative features, which is particularly important for distinguishing between visually similar hand gestures. As observed in the confusion matrix analysis, LeNet-5 struggles with fine-grained differences between certain classes, whereas the higher-capacity models are better equipped to capture these subtle variations.

V. Convolutional Autoencoder and Anomaly Detection

In addition to the supervised classification models, a convolutional autoencoder was implemented to explore an unsupervised approach to learning representations of ASL hand gesture images. Unlike the CNN models, which are trained with labelled data to directly predict the correct class, an autoencoder learns by attempting to reconstruct its own input. This makes it particularly well suited for anomaly detection, as a model trained exclusively on normal images will reconstruct them with low error, but will struggle to accurately reconstruct images that deviate significantly from what it has learned. The motivation for including this model was to demonstrate that meaningful structure can be extracted from the data without relying on labels, and to introduce a

method for identifying unusual or corrupted inputs that a standard classifier would not be able to detect.

The architecture consists of two stages. The encoder compresses each 28×28 grayscale image through three convolutional blocks with max pooling, progressively reducing the spatial dimensions from 28×28 down to a compact latent representation of 128 values. This bottleneck forces the model to retain only the most essential features of each hand sign. The decoder then reconstructs the image from this compressed representation using a series of transposed convolutional layers, restoring the output back to the original 28×28 size. The model was trained using mean squared error loss, which measures the pixel-level difference between the original and reconstructed image, and the Adam optimizer with an initial learning rate of 0.001. A learning rate scheduler and early stopping were applied, consistent with the approach used across the other models in this project. Training ran for all 30 epochs, with the best validation loss of 0.002273 achieved at the final epoch, indicating steady and stable convergence throughout.

For anomaly detection, reconstruction error was computed for each image in the test set. Clean test images produced a mean reconstruction error of 0.003463, with the 95th percentile used as the detection threshold, yielding a value of 0.006812. To evaluate the effectiveness of this approach, 200 synthetic anomalies were created by applying heavy salt-and-pepper noise and random block masking to real test images, simulating corrupted or out-of-distribution inputs. The results demonstrated strong detection performance: only 5.0% of clean images were incorrectly flagged as anomalous, while 100% of the corrupted images were correctly identified above the threshold. These results confirm that the autoencoder successfully learned the underlying structure of normal ASL hand gestures and can reliably distinguish them from corrupted inputs based on reconstruction error alone.

VI. Reflection and retrospective

Overall, this project provided a valuable and practical opportunity to apply machine learning and deep learning techniques to a real-world classification task. Working with the Sign Language MNIST dataset allowed the team to explore the full pipeline of a supervised learning project, from data preprocessing and exploratory analysis through to model design, training, evaluation, and comparison. The structured nature of the dataset meant that early results were encouraging, which helped the team build confidence in the implementation before moving on to more advanced components.

From a technical standpoint, one of the most effective aspects of the project was the decision to implement multiple architectures and evaluate them side by side. Training both LeNet-5 and the Custom CNN on identical data pipelines allowed for a direct and meaningful comparison, making it clear how architectural depth and modern regularization techniques such as batch normalization and dropout contribute to improved generalization. The Custom CNN achieving 98.65% test

accuracy compared to LeNet-5's 84.68% was a concrete and well-supported result that reinforced the theoretical concepts covered in the course. The implementation of the learning rate scheduler and early stopping also proved effective in practice, with training curves showing stable convergence across all models.

The addition of the convolutional autoencoder presented a different set of challenges and learning opportunities. As an unsupervised model, it required a shift in thinking away from labelled classification and towards reconstruction-based learning, which deepened the team's understanding of how neural networks can extract meaningful structure from data without supervision. Designing the encoder and decoder so that the spatial dimensions compressed and then recovered correctly required careful attention to the mathematics of convolutional and transposed convolutional layers. The anomaly detection results, where 100% of corrupted images were correctly flagged while only 5% of clean images were falsely flagged, demonstrated that the approach was sound and produced a reliable and interpretable outcome.

If the project were to be extended, potential improvements could include experimenting with transfer learning using pretrained models such as testing the system on real-world webcam images to evaluate robustness beyond the controlled dataset environment. Introducing additional traditional machine learning baselines such as SVM or Random Forest would also provide further context for evaluating the deep learning results. Nevertheless, the project successfully demonstrated the practical application of core deep learning concepts and produced strong, well-documented results across all implemented models.

VII. Conclusion

Overall, the comparison demonstrates that while LeNet-5 serves as a strong and efficient baseline, it is limited by its shallow architecture and lower representational capacity. In contrast, the custom CNN models achieve near-perfect classification performance due to their deeper structure and ability to learn more complex feature hierarchies. These results confirm that increasing model complexity can significantly enhance performance in sign language recognition tasks, particularly when fine-grained visual distinctions are required.